Deployment

English version of ../DEPLOYMENT_PART_B.md.

Start

Docker and Docker Compose are required. In the project root create `.env` from the sample and set `POSTGRES_PASSWORD`:

```
cp .env.example .env
docker compose up --build -d
docker compose exec api python -m backend.users create analyst@example.com
```

The interface is on port 8080, the API on 8000. State: `http://localhost:8000/health` and `/ready`.

Public hosting

In `.env` set the application addresses and enable secure cookies:

```
AUTH_ISSUER=https://app.example.com
AUTH_FRONTEND_LOGIN_URL=https://app.example.com/login
AUTH_COOKIE_SECURE=true
AEZA_CORS_ORIGINS=https://app.example.com
```

Terminate HTTPS at the reverse proxy and restrict public access to the API according to your network layout. Keep secrets only in `.env` or a secret manager.

Data and upgrades

On an empty PostgreSQL the application imports `dataset/data.json`. Another snapshot can be uploaded through the API; the schema reference is at `/api/v1/docs`.

Upgrading to a new version:

```
docker compose up --build -d
```

`docker compose down` stops the application and keeps the data. `docker compose down -v` deletes the database and account volumes.

Backup

The scripts save and restore both PostgreSQL and SQLite:

```
./scripts/backup.sh
./scripts/restore.sh backups/<directory>
./scripts/restore.sh backups/<directory> --yes
```

A restore without `--yes` only prints the plan. Verify a backup before relying on it.